

RELAZIONE DI PROGETTO: SISTEMA GESTIONE OSPEDALE

Docente: Prof. Porfirio Tramontana

Sviluppato da:

- **Vittorio Gaudiello (Matricola: DE1000046)**
- **Francesco Giannotti (Matricola: DE1000005)**

1. Introduzione e Obiettivi del Progetto

Il progetto consiste nello sviluppo di un'applicazione desktop in Java per la digitalizzazione e l'orchestrazione delle attività di una struttura ospedaliera. L'obiettivo principale è fornire un sistema integrato che permetta di gestire in modo efficiente il personale (Medici e Amministratori), le risorse fisiche (Reparti, Stanze, Letti) e il ciclo clinico dei pazienti (Ricoveri, Visite, Interventi Chirurgici). Il sistema è progettato per supportare accessi multi-utente con privilegi differenziati (Role-Based Access Control) interfacciandosi con un database relazionale (PostgreSQL) per garantire la persistenza, la consistenza e l'integrità dei dati.

2. Scelte Architetture e Design Pattern

Per garantire un codice modulare, manutenibile e scalabile, l'intero progetto è stato ingegnerizzato seguendo rigorosi standard di programmazione e i seguenti Design Pattern:

- **Pattern MVC (Model-View-Controller):**
 - **Model:** Rappresenta i dati e la logica di dominio (classi come Paziente, Ricovero, Medico). Ogni classe modella un'entità del database.
 - **View:** Gestita tramite Java Swing (MainGUI), si occupa esclusivamente di mostrare i dati all'utente e catturare gli input.
 - **Controller:** È il "cervello" dell'applicazione (Controller.java). Riceve le richieste dalla View, interroga i DAO per l'accesso al database, orchestra i dati in memoria (risolvendo i riferimenti incrociati degli oggetti) e aggiorna la View.
- **Pattern DAO (Data Access Object):** L'accesso al database è stato isolato in classi specifiche (AmministratoreImplementazioneDAO, MedicoImplementazioneDAO) che implementano un'interfaccia comune DAO. Questo disaccoppia la logica SQL dal resto dell'applicazione.

- **Pattern Singleton:** Applicato nella classe `ConnessioneDatabase` per garantire che l'applicazione istanzi una e una sola connessione al database PostgreSQL, ottimizzando il consumo di risorse ed evitando conflitti.

3. Modello dei Dati (Entità e Relazioni)

L'architettura del database e delle classi Java riflette fedelmente il dominio clinico, strutturato secondo le seguenti logiche relazionali:

- **Gerarchia Utenti:** È presente una superclasse `UtenteRegistrato` da cui ereditano `Amministratore` e `Medico`, favorendo il riuso del codice (dati anagrafici e credenziali).
- **Struttura Ospedaliera (Relazioni 1:N):** Un Reparto contiene molte Stanze; una Stanza contiene molti Letti. Il sistema impedisce di assegnare un letto se non è prima collocato in una stanza appartenente a un reparto.
- **Ciclo Clinico:** Il Paziente viene associato a un Ricovero (che impegna un Letto). Durante il ricovero possono essere effettuate delle Visite, dalle quali possono scaturire degli Interventi Chirurgici.
- **Risoluzione Relazioni Multi-a-Molti (N:N):** Per gestire i team medici e i pazienti assegnati, sono state modellate due classi/tabelle ponte:
 - **Gestisce:** Collega uno o più Medici a un Ricovero.
 - **Opera:** Collega uno o più Medici a un Intervento Chirurgico, specificandone anche il ruolo all'interno dell'equipe (es. Chirurgo, Anestesista).

4. Requisiti Funzionali e Attori (Use Cases)

L'applicativo prevede due attori principali con permessi nettamente separati, garantiti a livello di Controller e di query DAO.

- **Attore 1: Amministratore (Privilegi Massimi)**
 - **Operazioni CRUD:** Può aggiungere, modificare, visualizzare ed eliminare qualsiasi record nel database (Pazienti, Medici, Letti, Turni, ecc.).
 - **Orchestrazione:** Può collegare i medici ai turni lavorativi, assegnare i letti ai pazienti ricoverati e popolare le tabelle ponte (associando medici a interventi e ricoveri).
- **Attore 2: Medico (Modalità "Sola Lettura" Filtrata)**
 - **Sicurezza GDPR:** Il Medico non può inserire, modificare o cancellare record di sistema.
 - **Filtraggio Dati:** Grazie a query SQL parametrizzate, il medico visualizza una dashboard contenente esclusivamente i dati di sua competenza (i propri turni, i propri interventi programmati, i pazienti a lui assegnati e il proprio profilo anagrafico).

5. Manuale Utente e Interfaccia Grafica (GUI)

La vista utente è stata realizzata con `javax.swing` ed è ottimizzata per garantire la migliore User Experience (UX).

- **Login Dinamico:** La schermata di avvio discrimina l'ingresso tramite un menu a tendina. A seconda delle credenziali, il Controller rigenera la finestra principale (Dashboard) mostrando schede differenti.
- **JTabbedPane (Navigazione a Schede):** L'amministratore naviga tra 12 schede diverse. Le tabelle (JTable) si auto-ridimensionano in base ai dati.
- **Gestione Dinamica delle Relazioni (ComboBox):** In fase di inserimento (ad esempio di un nuovo Ricovero), l'utente non deve inserire codici numerici complessi a mano, ma seleziona il Paziente e il Letto da comodi menu a tendina che il Controller popola estraendo i nomi in tempo reale dal database.

The image shows two parts of the application interface. The top part is a login window titled 'Accesso al Sistema' with fields for 'Ruolo' (a dropdown menu), 'Email', 'Password', and an 'Accedi' button. The bottom part is a detailed view of the 'Ruolo' dropdown menu, showing 'Amministratore' as the selected option, with 'Amministratore' and 'Medico' as visible options. Below these is a horizontal navigation bar with tabs: 'Pazienti', 'Medici', 'Reparti', 'Stanze', 'Letti', 'Turni', 'Ricoveri', 'Interventi', 'Visite', 'Amministratori', 'Collega Med-Ricovero', and 'Collega Med-Intervento'.

Sincronizzazione in Real-Time: Ogni operazione di Salvataggio o Eliminazione attiva il metodo globale `rinfrescaDashboard()`. Questo garantisce che la GUI e il Database siano costantemente allineati, senza necessità di riavviare l'app.

6. Gestione delle Eccezioni e Validazione dei Dati

La robustezza del software è garantita da un rigoroso sistema di controlli a vari livelli:

- **Eccezioni Custom (BadArgsException):** Creata per intercettare parametri vuoti o non validi direttamente nei costruttori delle classi (Model), impedendo la creazione di oggetti corrotti in memoria.
- **Eccezioni SQL (SQLException):** Gestite nei DAO e propagate al Controller, impediscono il crash dell'app in caso di perdita di connessione, mostrando un pop-up di errore all'utente.
- **Formattazione Date (DateTimeParseException):** L'inserimento di turni e ricoveri richiede tempistiche precise. L'applicazione utilizza `LocalDateTime` e verifica che le

stringhe inserite nella GUI rispettino strettamente il formato yyyy-MM-dd HH:mm, bloccando salvataggi errati.

7. Dizionario dei Dati (Data Dictionary)

Per garantire una piena comprensione della struttura del **database** relazionale e degli oggetti in memoria, di seguito è riportato il dizionario dei dati che mappa le entità principali del sistema e i relativi attributi.

7.1 Utenti e Personale

- **UtenteRegistrato (Superclasse astratta):** Contiene i dati condivisi per l'autenticazione. Attributi: `nome`, `cognome`, `email`, `password`.
- **Amministratore:** Eredita da `UtenteRegistrato`. Identificato univocamente tramite `id_amministratore`.
- **Medico:** Eredita da `UtenteRegistrato`. Attributi specifici: `id_medico`, `tipo_medico` (indica la qualifica, es. Primario, Specialista), e una chiave esterna `id_reparto` che lo collega al reparto di appartenenza.

7.2 Struttura Ospedaliera

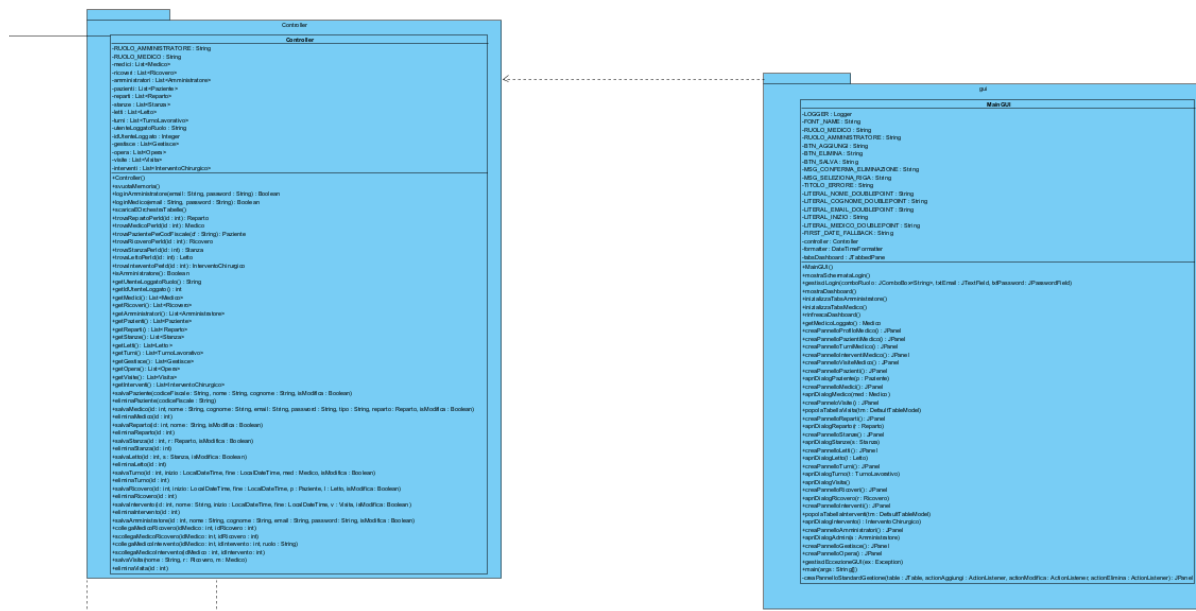
- **Reparto:** Modella le divisioni dell'ospedale. Attributi: `id_reparto` (autoincrementante), `nome_reparto`.
- **Stanza:** Unità fisica all'interno di un reparto. Attributi: `id_stanza`, `id_reparto` (chiave esterna).
- **Letto:** L'unità base per l'assegnazione dei pazienti. Attributi: `id_letto`, `id_stanza` (chiave esterna per rintracciare la collocazione fisica).

7.3 Ciclo Clinico

- **Paziente:** Modella i dati anagrafici dei ricoverati. Attributi: `cod_fiscale` (chiave primaria), `nome`, `cognome`.
- **Ricovero:** Traccia la permanenza del paziente. Attributi: `id_ricovero`, `data_ora_inizio`, `data_ora_fine` (formato `LocalDateTime`), `cod_fiscale` (paziente ricoverato), `id_letto` (letto occupato).
- **Visita:** Mappa gli esami effettuati durante un ricovero. Attributi: `id_visita`, `nome_visita`, `id_ricovero` (collegamento al ricovero), `id_medico` (medico referente).
- **Intervento Chirurgico:** Segue un'eventuale visita. Attributi: `id_intervento`, `nome_intervento`, `data_ora_inizio`, `data_ora_fine`, `id_visita` (la visita che ha generato l'intervento).

7.4 Associazioni Complesse (Tabelle Ponte N:N)

- **Gestisce (Medici - Ricoveri):** Collega i medici curanti ai vari ricoveri. Attributi: `id_medico`, `id_ricovero`.
- **Opera (Medici - Interventi):** Crea l'equipe chirurgica. Attributi: `id_medico`, `id_intervento`, `ruolo` (stringa che specifica la funzione del medico in sala operatoria).
- **Turno Lavorativo (Turni - Medici):** Assegna le fasce orarie lavorative. Attributi: `id_turno`, `data_ora_inizio`, `data_ora_fine`, `id_medico`



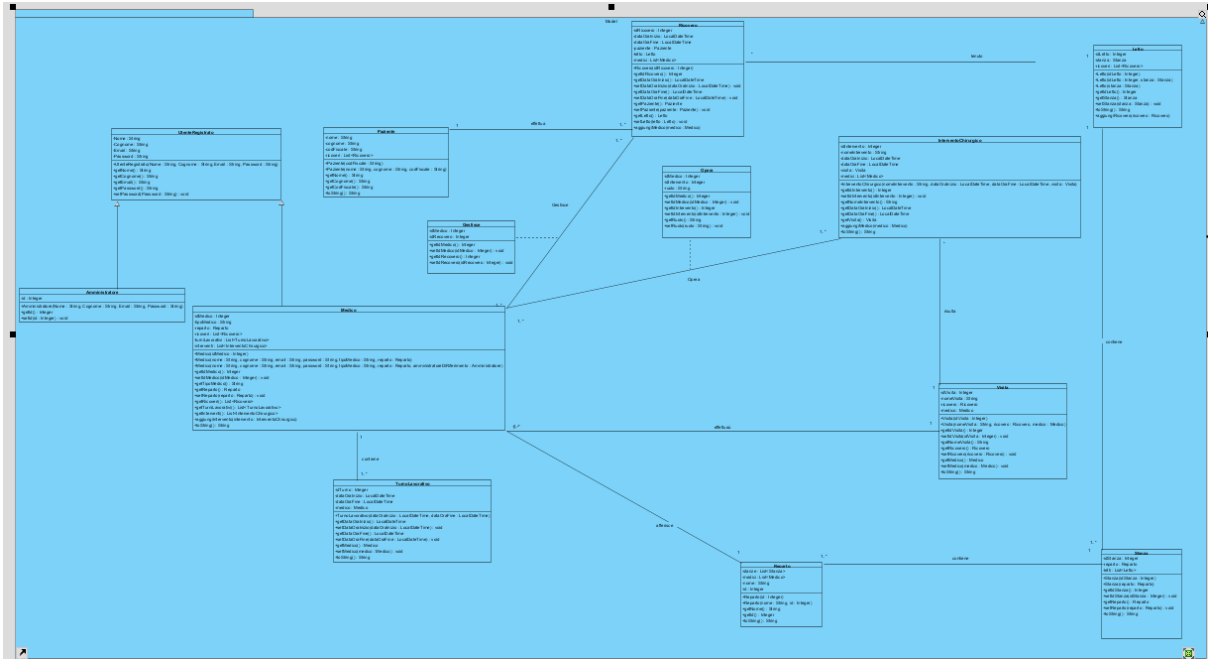
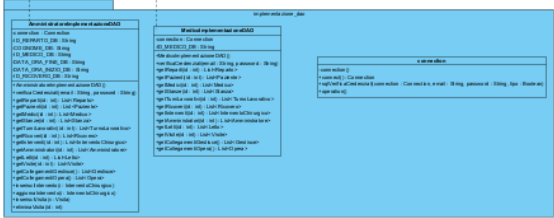


Diagramma di Sequenza - Fase di Login

